

LAPORAN PRAKTIKUM ALGORITMA PEMROGRAMAN
PERULANGAN WHILE DAN DO=WHILE



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH ALGORITMA PEMROGRAMAN

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 03 NOVEMBER 2025

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum **“Perulangan While dan Do=While ”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi hasil kegiatan praktikum yang bertujuan untuk memahami konsep perulangan while dan **do=while** di java dalam pemrograman. Penulis menyadari bahwa pemahaman mendalam terhadap topik ini merupakan fondasi essential bagi pengembangan keterampilan pemrograman yang lebih advanced.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. Bapak DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah algoritma pemograman, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.
2. Asisten praktikum yang telah membimbing dan membantu penulis selama sesi praktikum berlangsung.

Padang, 03 November 2025

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM ALGORITMA PEMROGRAMAN.....	i
KATA PENGANTAR	2
DAFTAR PUSTAKA.....	3
BAB I PENDAHULUAN.....	4
1. Latar Belakang.....	4
2. Tujuan	4
3. Manfaat	5
BAB II PEMBAHASAN	6
2.1 Landasan Teori.....	6
2.2 Langkah Pengerjaan.....	7
BAB III KESIMPULAN.....	14
3.1 Kesimpulan	14
3.2 Saran	14
DAFTAR PUSTAKA.....	15

BAB I

PENDAHULUAN

1. Latar Belakang

Sebagai mahasiswa pemula, saya sering kesulitan memahami algoritma hanya dari teori. Konsep baru benar-benar melekat justru saat saya mencobanya langsung lewat kode melihat bagaimana program bereaksi, mengulang, dan berhenti berdasarkan kondisi tertentu.

Berbeda dengan for yang cocok untuk jumlah perulangan tetap, perulangan while dan do=while digunakan saat kita tidak tahu pasti berapa kali suatu blok perlu dijalankan tetapi yang penting, terus berjalan selama kondisi masih terpenuhi. Keduanya mirip, tapi punya perbedaan krusial: while memeriksa kondisi dulu sebelum menjalankan kode, sedangkan do=while menjalankan kode dulu, baru mengecek kondisi. Artinya, do=while pasti jalan minimal sekali, apa pun kondisinya.

Perbedaan ini kecil, tapi berdampak besar seperti misalnya saat membuat validasi input pengguna. Kita pasti ingin pengguna memasukkan data minimal sekali sebelum dicek, dan di sinilah do=while jadi pilihan yang lebih tepat.

Praktikum ini bertujuan untuk mengasah pemahaman saya tentang kapan dan mengapa memilih while atau do=while, bukan hanya dari aturan sintaks, tapi dari cara kerjanya dalam skenario nyata. Dengan menulis dan mengamati langsung hasil eksekusinya, saya berharap bisa membangun logika yang lebih kuat dalam menangani masalah berulang berbasis kondisi.

2. Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Mengetahui dan mengaplikasikan penggunaan statement while dan do=while di java.
2. Mampu menggunakan struktur while dan do=while untuk mengeksekusi blok saat kondisi tidak diketahui.

3. Melatih kemampuan berpikir logis dalam menyelesaikan masalah yang bersifat repetitif tanpa menulis kode secara manual berulang kali.

3. Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Melatih logika pemrograman melalui pembuatan pola, pengulangan tugas, dan pengolahan data sederhana saat kondisi tidak diketahui.
2. Membiasakan diri memilih struktur perulangan yang tepat sesuai kebutuhan logika program, bukan hanya menyalin sintaks

BAB II

PEMBAHASAN

2.1 Landasan Teori

Dalam pemrograman, tidak semua tugas bisa diselesaikan dengan jumlah pengulangan yang pasti. Sering kali, kita perlu mengulang suatu proses selama suatu kondisi tertentu masih terpenuhi, tanpa tahu pasti berapa kali itu akan terjadi. Untuk kebutuhan seperti ini, Java menyediakan dua jenis perulangan berbasis kondisi: `while` dan `do=while`.

Keduanya termasuk dalam kategori `uncounted loop` karena jumlah iterasinya tidak ditentukan sejak awal, melainkan tergantung pada perubahan nilai dalam proses eksekusi. Namun, meski mirip, keduanya berbeda dalam urutan pengecekan kondisi.

- Perulangan `while`

Menurut Pemburukode (2025), perulangan `while` mengevaluasi kondisi sebelum menjalankan blok kode. Jika kondisi bernilai `true`, blok kode dijalankan; jika `false`, blok tersebut dilewati sama sekali. Struktur umumnya sebagai berikut:

```
while (kondisi) {  
    // blok kode yang diulang  
}
```

Karena pengecekan dilakukan di awal, ada kemungkinan blok kode tidak pernah dijalankan, terutama jika kondisi awal sudah bernilai `false`.

- Perulangan `do=while`

Berbeda dengan `while`, perulangan `do=while` (menurut Petanikode, 2025) menjalankan blok kode terlebih dahulu, baru kemudian mengevaluasi kondisi di akhir. Hal ini menjamin bahwa blok kode di dalam `do=while` akan dijalankan minimal satu kali, bahkan jika kondisi awalnya `false`. Struktur umumnya:

```
do {  
    // blok kode yang diulang  
} while (kondisi);
```

Perbedaan ini membuat do=while cocok digunakan dalam kasus yang memerlukan eksekusi awal sebelum validasi, misal kita meminta input pengguna dan baru kemudian memeriksa apakah input tersebut valid.

2.2 Langkah Pengerjaan

a. Perulangan While 1

1. Buat package dan *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan while dengan format umum seperti ini :

```
while (kondisi) {  
    // blok kode yang diulang  
}
```

3. Pada program ini saya membuat Program yang digunakan untuk menghitung berapa kali pengguna melakukan perulangan, dengan keputusan lanjut atau berhenti ditentukan oleh input pengguna.
4. Scanner digunakan untuk agar pengguna bisa memasukkan input iya/tidak.

```
package Pekan6_2511532017;  
  
import java.util.Scanner;  
  
public class perulanganWhile1_2511532017 {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        int counter=0;  
        String jawab;  
        boolean running = true;  
        //deklrasi scanner  
        Scanner scan = new Scanner(System.in);  
        while (running) {  
            counter++;  
            System.out.println("Jumlah = "+ counter);  
            System.out.print("Apakah lanjut (ya/tidak?) ");  
            jawab= scan.nextLine();  
            //cek jawab = tidak ,perulangan berhenti  
            if ( jawab.equalsIgnoreCase("tidak")) {  
                running=false;  
            }  
        }  
        System.out.println("Anda sudah melakukan perulangan sebanyak "+ counter++);  
    }  
}
```

Gambar 2.2. 1

5. Perulangan dimulai dengan kondisi while (running), artinya selama running bernilai true, blok kode di dalamnya akan terus dijalankan.
6. Di awal setiap iterasi, nilai counter ditambah 1 (counter++) untuk mencatat bahwa satu putaran perulangan telah terjadi.
7. Program menampilkan nilai counter saat ini.

8. Pengguna diminta memasukkan jawaban: “ya” untuk lanjut, “tidak” untuk berhenti.
9. Jika pengguna memasukkan nilai “tidak” maka running (boolean) akan bernilai false dan berhenti.
10. Dengan output sebagai berikut :

```
Jumlah = 1
Apakah lanjut (ya/tidak?) ya
Jumlah = 2
Apakah lanjut (ya/tidak?) ya
Jumlah = 3
Apakah lanjut (ya/tidak?) ya
Jumlah = 4
Apakah lanjut (ya/tidak?) ya
Jumlah = 5
Apakah lanjut (ya/tidak?) ya
Jumlah = 6
Apakah lanjut (ya/tidak?) ya
Jumlah = 7
Apakah lanjut (ya/tidak?) ya
Jumlah = 8
Apakah lanjut (ya/tidak?) tidak
Anda sudah melakukan perulangan sebanyak 8
```

Gambar 2.2. 2

b. Sentinel Loop

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan while dengan format umum seperti ini :

```
while (kondisi) {

    // blok kode yang diulang

}
```

3. Pada program ini saya membuat Program yang digunakan untuk menghitung penjumlahan berdasar input pengguna, jika pengguna memasukkan angka 0 maka program akan berhenti dan menampilkan hasil penjumlahan.
4. Scanner digunakan untuk agar pengguna bisa masukkan angka yang akan dijumlahkan.

```

package Pekan6_2511532017;

import java.util.Scanner;

public class SentinelLoop_2511532017 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Scanner console = new Scanner (System.in);
        int sum =0;
        int number= 12; //dummy value , anything but 0

        while (number != 0) {
            System.out.print("Masukkan angka (0 untuk keluar) : ");
            number = console.nextInt();
            sum = sum + number;
        }
        System.out.println("totalnya adalah " + sum);
    }
}

```

Gambar 2.2. 3

5. Number =12 digunakan sebagai dummy value agar kode dapat (bernilai false) berjalan karena bukan 0
6. Jika pengguna memasukkan angka selain 0 maka angka akan ditambahkan ke sum.
7. Jika pengguna memasukkan angka 0 maka program akan berhenti berjalan (bernilai true).
8. Berikut output dari programnya :

```

Masukkan angka (0 untuk keluar) : 49
Masukkan angka (0 untuk keluar) : 51
Masukkan angka (0 untuk keluar) : 0
totalnya adalah 100

```

Gambar 2.2. 4

c. Lempar Dadu

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan while dengan format umum seperti ini :

```

while (kondisi) {

    // blok kode yang diulang

}

```

3. Pada program ini saya membuat Program sederhana permainan lempar dadu, dimana dadu1 dan dadu2 akan dirandom dan kedua dadu tersebut akan dijumlahkan dan menghasilkan angka 7 (kondisi menang). Jika hasil penjumlahan tidak menghasilkan angka 7 maka kedua dadu akan dirandom hingga mendapatkan penjumlahan dengan hasil 7.
4. Import java.util.Random digunakan untuk agar angka random dapat dipanggil.

```
package Pekan6_2511532017;

import java.util.Random;

public class Lempardadu_2511532017 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Random rand= new Random();
        int tries = 0;
        int sum = 0;
        while (sum !=7) {
            //roll the dice once
            int dadu1 = rand.nextInt(6)+1;
            int dadu2 = rand.nextInt(6)+1;
            sum = dadu1 + dadu2;
            System.out.println(dadu1 + " + " + dadu2 + " = " + sum);
            tries++;
        }
        System.out.println("You won after " + tries + " tries!");
    }
}
```

Gambar 2.2. 5

5. Perulangan berjalan terus jika penjumlahan tidak sama dengan 7 (sum!=7)
6. Dadu1 dan dadu2 di acak lalu nilai keduanya dijumlahkan.
7. Hasil penjumlahan akan ditampilkan, jika penjumlahan tidak menghasilkan angka 7 maka dilakukan perulangan lagi.
8. Jumlahan percobaan dinaikkan jika hasil penjumlahan tidak menghasilkan angka7.
9. Output berupa :

```
2 + 3 = 5
2 + 6 = 8
5 + 2 = 7
You won after 3 tries!
```

Gambar 2.2. 6

d. Game Penjumlahan

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.

- Setelah *class* dibuat, buat struktur perulangan *while* dengan format umum seperti ini :

```
while (kondisi) {  
    // blok kode yang diulang  
}
```

- Pada program ini saya membuat Program sederhana tentang permainan penjumlahan. Dimana akan ditampilkan soal lalu kita harus menjawabnya dan memiliki 3 kesempatan untuk salah.
- Import *java.util.Random* digunakan untuk agar angka random dapat dipanggil.

```
package Pekan6_2511532017;  
  
import java.util.Random;  
import java.util.Scanner;  
  
public class GamePenjumlahan_2511532017 {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Scanner console = new Scanner (System.in);  
        Random rand = new Random();  
        //play until user gets 3 wrong  
        int points= 0;  
        int wrong = 0;  
        while (wrong <3) {  
            int result = play(console,rand);  
            if(result >0) {  
                points++;  
            }else {  
                wrong++;  
            }  
        }  
        System.out.println("You earned " +points + " total points.");  
    }  
    //membuat soal penjumlahan dan ditampilkan ke user  
    public static int play (Scanner console, Random rand) {  
        //print the operands being added, and sum them  
        int operands= rand.nextInt(4) + 2;  
        int sum = rand.nextInt(10) + 1;  
        System.out.print(sum);  
        for (int i = 2; i <= operands; i++) {  
            int n = rand.nextInt(10) + 1;  
            sum += n;  
            System.out.print(" + " + n);  
        }System.out.print(" = ");  
  
        // read user's guess and report whether it was correct  
        int guess = console.nextInt();  
        if (guess == sum) {
```

Gambar 2.2. 7

```

        return 1;
    }else {
        System.out.println("Wrong! the answer was " + sum);
        return 0;
    }
}
}

```

Gambar 2.2. 8

5. Program akan terus berjalan jika kesalahan kurang dari 3.
6. Di setiap putaran, program memanggil metode play() untuk menghasilkan soal dan menilai jawaban pengguna.
7. Soal hanya akan menampilkan 2 sampai 5 operasi dan angka yang akan dioperasikan memiliki rentang 1 sampai 10 saja.
8. Jika jawaban benar maka soal akan diberikan kembali dan point dihitung (points++).
9. Jika salah maka (wrong++) dan jawaban yang benar akan ditampilkan.
10. Jika wrong = 3 maka kesempatan sudah habis dan akan dicetak total points.
11. Output berupa :

```

9 + 3 + 3 + 7 + 6 = 28
4 + 2 + 2 + 10 = 18
10 + 2 + 5 + 6 = 23
9 + 6 + 10 + 2 + 2 = 29
1 + 2 = 3
8 + 8 + 10 = 1
Wrong! the answer was 26
4 + 6 + 2 = 1
Wrong! the answer was 12
10 + 3 + 1 + 8 + 6 = 4
Wrong! the answer was 28
You earned 5 total points.

```

Gambar 2.2. 9

e. Do While

1. Buat *class* baru untuk pembuatan file yang bisa dilakukan untuk operasi pemrograman Java.
2. Setelah *class* dibuat, buat struktur perulangan do=while dengan format umum seperti ini :

```

do {
    // blok kode yang diulang
} while (kondisi);

```

3. Pada program ini saya membuat Program untuk memasukkan pasword berulang kali hingga password yang dimasukkan benar.
4. Scanner digunakan agar user bisa memasukkan password.

```
package Pekan6_2511532017;

import java.util.Scanner;

public class dowhile1_2511532017 {

    public static void main(String[] args) {
        Scanner console = new Scanner (System.in);
        String phrase;
        do {
            System.out.print("Input Password: ");
            phrase = console.next();
        } while (!phrase.equals("abcd"));

    }
}
```

Gambar 2.2. 10

5. Program langsung meminta input dari user (do).
6. Setelah input dimasukkan maka input akan dicek ,apakah sama dengan (abcd).
7. Jika password salah (true) maka akan diulang untuk memasukkan password.
8. Tetapi jika password benar (false) maka kode akan berhenti.
9. Dengan ouput sebagai berikut :

```
Input Password: eeee
Input Password: abcd
```

Gambar 2.2. 11

BAB III

KESIMPULAN

3.1 Kesimpulan

Praktikum perulangan `while` dan `do=while` memberikan pemahaman langsung tentang cara menangani tugas berulang yang jumlahnya tidak pasti di awal. Keduanya sangat berguna dalam situasi nyata, seperti validasi input pengguna, menu interaktif, atau proses yang bergantung pada kondisi dinamis. Dengan mempraktikkan keduanya, kita belajar membedakan kapan suatu blok kode harus dijalankan minimal sekali (`do=while`) dan kapan cukup dicek dulu sebelum dijalankan (`while`). Pemahaman ini membantu menulis program yang lebih logis, aman dari infinite loop, dan sesuai dengan alur masalah yang dihadapi.

3.2 Saran

Untuk meningkatkan pemahaman yang lebih mendalam tentang perulangan `while` dan `do=while`, maka dari itu ada beberapa saran yang bisa dilakukan:

- Perbanyak latihan dengan berbagai skenario nyata, seperti validasi input, menu interaktif, atau simulasi permainan sederhana, agar lebih peka terhadap perbedaan penggunaan `while` dan `do-while` dalam konteks logika program.
- Latih diri untuk menganalisis terlebih dahulu apakah suatu blok kode perlu dijalankan minimal sekali atau tidak sebelum memilih jenis perulangan, hal ini akan memperkuat kemampuan dalam merancang alur program yang efisien dan tepat.

Masih banyak kekurangan dalam pembuatan laporan praktikum ini, maka dari itu penulis sangat terbuka untuk menerima saran dari pembaca sekalian.

DAFTAR PUSTAKA

- [1] Petanikode, “Perbedaan Perulangan While dan Do/While,” *Petanikode*, 2025. [Online]. Available: <https://www.petanikode.com/perbedaan-perulangan-while-dan-do-while/>. [Accessed: Nov. 3, 2025].
- [2] Pemburukode, “Perulangan Do While Java: Pengenalan dan Contoh Penggunaan,” *Pemburukode*, 2025. [Online]. Available: <https://pemburukode.com/perulangan-do-while-java/>. [Accessed: Nov. 3, 2025].